ANALISIS DAN IMPLEMENTASI MEKANISME KRIPTOGRAFI KRIPTOGRAFI ASIMETRIS (ASYMMETRIC) RIVEST, SHAMIR, AND ADLEMAN (RSA)

1. Pengantar Singkat
   - Sejarah RSA
     RSA dikembangkan pada tahun 1977 oleh tiga ilmuwan yaitu : Ron Rivest, Adi Shamir, dan Leonard Adleman. Nama RSA diambil dari huruf awal ketiga penemunyya. RSA merupakan algoritma kriptografi asimetris yang menggunakan dua kunci berbeda yaitu kunci public (untuk mengenkripsi pesan) dan kunci privat (untuk mendekripsi pesan).
   - Kegunaan di Dunia Nyata
     RSA digunakan dalam : HTTPS (Secure Web Communication), Digital Signature, SSL/TLS, Pengamanan Email, Sistem Perbankan. Keamanan RSA didasarkan pada kesulitan memfaktorkan bilangan besar menjadi faktor primanya.
   - Konsep dasar RSA
     Keamanan RSA didasarkan pada satu fakta penting dalam matematika : Sangat mudah mengalikan dua bilangan prima besar, tetapi sangat sulit memfaktorkan hasil perkaliannya kembali menjadi bilangan prima semula. Selain bergantung pada kesulitan faktorisasi bilangan besar, RSA juga menggunakan konsep aritmetika modular dan Teorema Euler. Dalam aritmetika modular, operasi perpangkatan dilakukan dengan mengambil sisa hasil pembagian terhadap suatu bilangan (modulus). Secara matematis, RSA bekerja karena memenuhi sifat :
     Jika gcd(M, n) = 1, maka berdasarkan Teorema Euler :
     $$M^{\varphi(n)} \equiv 1 \pmod{n}$$
     Karena nilai e dan d dipilih sehingga :
     $$e \times d \equiv 1 \pmod{\varphi(n)}$$
     Maka berlaku :
     $$M^{ed} \equiv M \pmod{n}$$
     Inilah alasan secara matematis mengapa pesan yang dienkripsi dapat dikembalikan ke bentuk semula.

2. Pembangkitan Kunci (Key Generation)
   Langkah-langkah :
   - Pilih dua bilangan prima besar : p dan q
   - Hitung :
     $n = p \times q$
     n disebut modulus
   - Hitung fungsi totien Euler :
     $\varphi(n) = (p - 1)(q - 1)$

Nilai e harus relatif prima terhadap φ(n) agar memiliki invers modular. Invers modular ini diperlukan untuk mencari nilai d, yaitu bilangan yang memenuhi :

$$d \times e \equiv 1 \pmod{\varphi(n)}$$

Nilai d dihitung menggunakan Extended Euclidean Algorithm, yaitu algoritma untuk mencari solusi persamaan linear dalam bentuk :

$$ax + by = gcd(a, b)$$

Karena gcd(e, φ(n)) = 1, maka solusi invers modular pasti ada.

- Pilih bilangan e sehingga :
  1 < e < φ(n)
  gcd(e, φ(n)) = 1
- Hitung d :
  d adalah invers modular dari e terhadap φ(n)
  d × e ≡ 1 (mod φ(n))

Hasil :

- Public Key = (e, n)
- Private Key = (d, n)

Distribusi :

- Public key boleh dibagikan ke siapa saja
- Private key harus dirahasiakan

3. Proses Enkripsi : mengubah *Plaintext* (data dalam bentuk bisa dibaca) menjadi *Ciphertext* (data acak atau tidak bermakna).
   - Misalkan :
     Plaintext = M
   - Rumus enkripsi :
     C = M^e mod n
     Proses perpangkatan dalam RSA tidak dilakukan secara langsung karena nilai e dan n biasanya sangat besar. Oleh karena itu digunakan teknik modular exponentiation, yaitu metode efisien untuk menghitung perpangkatan dalam modulo tanpa menghasilkan angka yang terlalu besar. Operasi ini dapat ditulis sebagai:
     $$C = M^e \mod n$$
     Dimana hasil akhir selalu berada dalam rentang 0 hingga n−1.
   - Langkah :
     Ubah pesan menjadi angka (misal ASCII)
     Pangkatkan dengan e
     Modulo dengan n
     Hasilnya adalah Ciphertext (C)

4. Proses Dekripsi : mengembalikan *Ciphertext* menjadi *Plaintext*.
   - Rumus dekripsi :
     M = C^d mod n

     Dekripsi bekerja karena hubungan matematis antara e dan d. Karena :

$$e \times d \equiv 1 \pmod{\varphi(n)}$$

Maka terdapat suatu bilangan k sehingga:

$$ed = 1 + k\varphi(n)$$

Sehingga saat dilakukan dekripsi :

$$C^d = (M^e)^d = M^{ed}$$

$$M^{ed} = M^{1+k\varphi(n)}$$

$$M^{1+k\varphi(n)} = M \times (M^{\varphi(n)})^k$$

Berdasarkan Teorema Euler :

$$M^{\varphi(n)} \equiv 1 \pmod{n}$$

Sehingga :

$$M^{ed} \equiv M \pmod{n}$$

Inilah alasan secara matematis pesan kembali ke bentuk semula.

- Langkah :
  Ambil ciphertext C
  Pangkatkan dengan d
  Modulo dengan n
  Hasilnya kembali menjadi plaintext

5. Kelebihan dan Kelemahan
   Kelebihan :
   - Sistem public key → aman untuk distribusi kunci
   - Digunakan luas dalam protokol internet
   - Keamanan tinggi jika ukuran kunci besar (2048-bit ke atas)

   Kelemahan :
   - Lambat dibanding kriptografi simetris
   - Rentan jika ukuran kunci kecil
   - Bergantung pada kesulitan faktorisasi bilangan besar
   - Tidak tahan terhadap komputer kuantum

Link Github :
https://github.com/006mareta/TugasKripto_Mareta-Surya-Tananjaya_24051204006_RSA